

# Four Week Training Program for Vision AI

## Learning Objectives

By the end of this 4-week program, you will be able to:

- Understand the fundamentals of deep learning and neural networks.
- Build, train, and evaluate Convolutional Neural Networks (CNNs) using Python and PyTorch.
- Implement common computer vision tasks, including image classification, object detection, and segmentation.
- Apply transfer learning with pre-trained models to solve custom problems efficiently.
- Develop and containerize a real-time vision application, leveraging your experience with low-latency systems.

## Core Technologies

- **Primary Language:** Python
  - **Key Libraries:** PyTorch, OpenCV, NumPy, Matplotlib
  - **Tools:** Jupyter Notebooks, Git, Docker
- 

## Week 1: AI & Deep Learning Foundations

This week establishes the theoretical and practical groundwork, transitioning from your classical ML experience to deep learning.

- **Days 1-2: From ML to Deep Learning**
  - **Topics:** Review of Machine Learning concepts (supervised vs. unsupervised). Introduction to neural networks, activation functions, backpropagation, and gradient descent.
  - **Goal:** Understand the core mechanics of how neural networks learn.
- **Days 3-4: Introduction to PyTorch**
  - **Topics:** Setting up the PyTorch environment. Mastering Tensors, automatic differentiation with `autograd`, and building sequential models with `torch.nn`.
  - **Practical:** Build a simple neural network to solve a basic regression or classification problem on tabular data.
- **Days 5-7: Image Processing Fundamentals with OpenCV**

- **Topics:** Reading, writing, and manipulating images. Color spaces, transformations (scaling, rotation), filtering, and edge detection.
- 

## Week 2: Convolutional Neural Networks (CNNs)

This week dives into the core architecture for modern computer vision.

- **Days 1-3: Understanding CNNs**
    - **Topics:** The convolution operation, pooling layers, padding, and stride. Understanding how CNNs learn hierarchical features from images.
    - **Practical:** Build a CNN from scratch in PyTorch to classify images from the MNIST or CIFAR-10 dataset.
  - **Days 4-5: Training, Evaluation & Optimization**
    - **Topics:** Loss functions, optimizers, learning rate scheduling, and overfitting/underfitting. Techniques like data augmentation and regularization.
    - **Practical:** Fine-tune your CNN from the previous days, implementing data augmentation to improve its accuracy.
  - **Days 6-7: Transfer Learning with Pre-trained Models**
    - **Topics:** The concept of transfer learning. Exploring famous architectures (e.g., ResNet, VGG, MobileNet) and using them as feature extractors or for fine-tuning.
    - **Goal:** Learn the most effective and common approach for building high-performance vision models without training from scratch.
- 

## Week 3: Advanced Vision Tasks

Go beyond simple classification to solve more complex and practical problems.

- **Days 1-3: Object Detection**
  - **Topics:** Understanding bounding boxes, Intersection over Union (IoU), and non-maximum suppression. Introduction to popular object detection architectures like YOLO (You Only Look Once).
  - **Practical:** Use a pre-trained YOLO model to perform real-time object detection on a video stream from a webcam. This will leverage your experience with real-time data processing.
- **Days 4-5: Image Segmentation**
  - **Topics:** Differentiating between semantic and instance segmentation. Introduction to architectures like U-Net.

- **Practical:** Use a pre-trained segmentation model to identify and mask specific objects in images.
  - **Days 6-7: Introduction to Video Analysis**
    - **Topics:** Object tracking, action recognition, and processing video data efficiently.
    - **Goal:** Combine your knowledge of real-time data streams and vision models to understand how to build applications that interpret video.
- 

## Week 4: Capstone Project & Deployment

This week is dedicated to applying all the learned concepts to a single, comprehensive project.

- **Project Idea: Real-Time Personal Protective Equipment (PPE) Detector**
  - This project involves building a system that can detect whether a person in a video feed is wearing a hard hat and safety vest. It's a practical application with clear real-world value.
- **Days 1-4: Project Development**
  - **Task 1: Data Collection & Preparation:** Find an open-source dataset of people with and without PPE.
  - **Task 2: Model Training:** Fine-tune a pre-trained YOLO model on your custom dataset.
  - **Task 3: Application Building:** Write a Python script using OpenCV to capture a video feed, run inference with your custom model frame-by-frame, and draw bounding boxes on the screen.
- **Days 5-6: Optimization & Deployment**
  - **Task 4: Containerization:** Leverage your DevOps knowledge by creating a `Dockerfile` to containerize your application and all its dependencies.
  - **Task 5: Deployment:** Deploy the containerized application locally.
- **Day 7: Review and Future Steps**
  - **Goal:** Document your project on GitHub, reflecting on the challenges and outcomes. Outline potential next steps in your AI journey, such as exploring Generative AI (Stable Diffusion, GANs) or Vision Transformers (ViT).